ECE 285 Final Project Report: Agentic EV Charging Scheduling Assistant

Ryan Luo
UC San Diego

Peter Quawas
UC San Diego

Abstract

We build and evaluate AGENTICEV, an agentic assistant for day-ahead EV charging scheduling using real Caltech ACN-Data sessions. Given session arrival/departure windows, requested energy, per-session power limits, and a 50 kW site cap, the system produces a feasible schedule minimizing time-of-use (TOU) cost. We compare three approaches: (i) a CVXPY convex optimizer, (ii) a single-shot GPT-4o LLM baseline, and (iii) AGENTICEV, which uses GPT-4o to call the optimizer as a verified tool and generate grounded explanations. Over 19 benchmark days, AGENTICEV exactly matches the optimizer on all metrics. The LLM baseline's lower reported cost is an artifact of under-delivery: $12\times$ more unmet kWh and $6.6\times$ more checker flags, including hard physical-constraint violations that the solver and agent avoid entirely.

1 Introduction and Related Work

Motivation. At the Caltech ACN-Data JPL site, peak EV demand can reach 200 kW—nearly $4\times$ the 50 kW site cap—meaning unmanaged charging regularly exceeds limits and raises costs under TOU tariffs. A practical scheduler must handle time-varying prices, overlapping session windows, and per-session power limits [3, 5]. Convex optimization solves this reliably [3], but requires precisely structured inputs and returns no natural-language output. LLMs accept free-form queries [1], but fail on constrained numerical programs. We investigate whether routing natural-language requests through an LLM to a convex solver can deliver both the reliability of optimization and the accessibility of natural-language interaction.

Task. Given a natural-language description of a day's sessions, produce $p_{i,t}$ [kW] that (1) satisfies all physical constraints, (2) minimizes TOU cost, and (3) delivers requested energy wherever the site cap allows.

Contributions.

- **Method:** AGENTICEV maps NL requests to optimizer inputs, calls CVXPY, and generates explanations from tool-returned metrics—structurally preventing numeric hallucination.
- **Evaluation:** A shared pipeline runs all three methods on identical session data, scored by the same constraint checker and metrics code.
- **Outcome:** AGENTICEV matches the optimizer exactly over 19 days; the LLM baseline's apparent cost savings reflect under-delivery, not genuine optimization.

Related work. EV scheduling has been studied at large scale with network constraints [5, 3]. The ReAct framework [6] shows tool-augmented LLMs can improve reliability on structured tasks over single-shot generation. We apply these ideas to EV scheduling, delegating all numerical computation to a certified external solver and thereby providing verifiability guarantees not achievable through LLM reasoning alone.

2 Problem Formulation

Notation. Sessions $\mathcal{I}$ are scheduled over $\mathcal{T} = \{0, \ldots, 95\}$ (15-min steps, one day). Each session $i \in \mathcal{I}$ has: arrival a_i, departure d_i (half-open window $[a_i, d_i)$), requested energy E_i^{req} [kWh], and max power $\bar{p}_i$ [kW]. The site has aggregate cap $\bar{P}_t$ [kW] and TOU price c_t [\$/kWh]. Decision variables: power $p_{i,t}$ [kW] and per-session unmet slack $u_i \geq 0$ [kWh].

Preprint.

Core formulation.

$$\min_{\{p_{i,t}\},\{u_i\}} \Delta t \sum_t c_t \sum_i p_{i,t} + \lambda \sum_i u_i \tag{1}$$

$$\text{s.t.} \quad p_{i,t} = 0, \quad t \notin [a_i, d_i) \quad \text{(availability)} \tag{2}$$

$$0 \le p_{i,t} \le \bar{p}_i \quad \text{(per-session power)} \tag{3}$$

$$\textstyle\sum_i p_{i,t} \le \bar{P}_t \quad \text{(site cap)} \tag{4}$$

$$\Delta t \sum_{t=a_i}^{d_i-1} p_{i,t} + u_i = E_i^{\text{req}} \quad \text{(energy balance)} \tag{5}$$

$$u_i \ge 0 \tag{6}$$

With $\Delta t = 0.25$ h and $\lambda = 10^6$ \$/kWh, the program is linear (hence convex) and solved to global optimality by CVXPY (default solver backend). The large penalty ensures $u_i > 0$ only when the site cap makes full delivery physically impossible. All constraints are independently re-verified by the shared constraint checker after solving. The checker reports two categories of flags: *hard constraint violations* on (2)–(4) (physical infeasibility) and *energy shortfall flags* where $u_i > 0$ (under-delivery forced by the site cap).

Success criteria. A run is **successful** if: (1) the schedule has zero hard constraint violations (tol 10^{-5}) on (2)–(4); energy shortfall flags ($u_i > 0$) are permitted when the site cap makes full delivery physically impossible; (2) cost, peak, unmet energy, and sessions served match the CVXPY reference on identical data; and (3) every number in the agent's explanation matches the corresponding field in the tool's JSON output.

3 Methodology

3.1 System Overview

Raw ACN-Data sessions are preprocessed into a standardized table (step-indexed arrival/departure, energy, max power) that feeds three end-to-end pipelines, all sharing the same evaluation layer (Figure 1):

1. **Phase A — Numerical solver:** Direct CVXPY solution of (1)–(6). No LLM involved.
2. **Phase B — LLM-only baseline:** A single GPT-4o call receives the full problem as text and outputs the $p_{i,t}$ matrix directly. No optimizer or checker is accessible during generation.
3. **Phase C — AGENTICEV:** GPT-4o parses the NL request, calls `solve_ev_schedule` (CVXPY wrapper) and constraint checker as OpenAI function-calling tools in a multi-turn loop. Then writes an explanation grounded in the returned metrics.

3.2 Data, Environment, and Tools

We use the Caltech ACN-Data JPL site [2] (50 kW cap, 96×15-min steps per day). Raw timestamps are mapped to step indices; sessions clipped to the day boundary. Missing max-power values filled from charger metadata. Two tools are shared:

- **CVXPY optimizer:** Solves (1)–(6) with $\lambda = 10^6$. Returns optimal schedule, cost, peak, and per-session unmet energy.
- **Constraint checker:** Independently verifies all constraint types (availability, per-session power, site cap, energy over/under-delivery), returns a flag list distinguishing hard constraint violations from energy shortfall flags, and computes all evaluation metrics. Each type is covered by unit tests.

In Phase C the optimizer is exposed via the OpenAI function-calling API whereas in Phase A it is called directly. In Phase B neither tool is accessible during generation.

3.3 Agent Design and Model/Tool Interaction

AGENTICEV follows the ReAct pattern [6] in a structured five-step loop (Table 1):

1. **Parse (GPT-4o):** Converts the NL request to structured session parameters—step-indexed arrival/departure windows, per-session energy and max power. Time expressions (e.g., "6 pm") are mapped to step indices (e.g., step 72).
2. **Optimize (CVXPY tool):** Calls `solve_ev_schedule`, which runs the convex solver and returns a JSON summary: `total_cost_usd`, `peak_load_kw`, `total_unmet_kwh`, `pct_fully_served`.

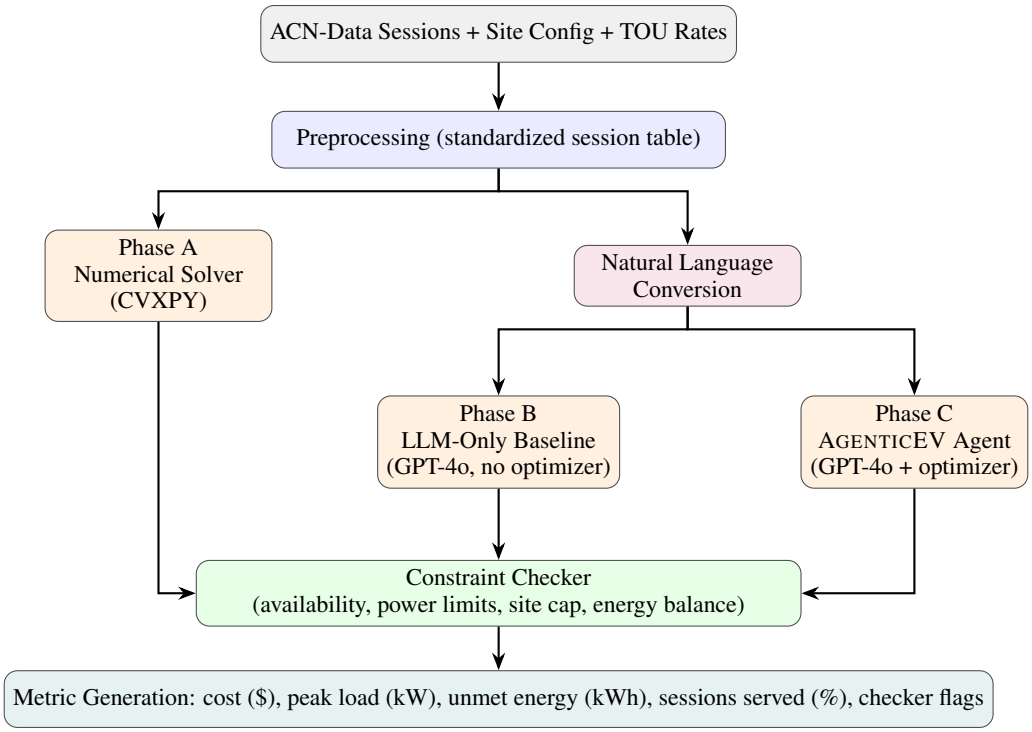

Figure 1: System architecture. All three pipelines share a constraint checker and metrics layer. Phase A receives the structured session table directly; Phases B and C convert it to natural language first. Only Phase C has tool access to the optimizer during generation.

3. **Validate (checker):** The returned schedule is passed to the constraint checker and any flags are surfaced to the agent.
4. **Refine (GPT-4o):** On parse error or tool failure, GPT-4o corrects the structured input and retries. The loop completes within 3 rounds on every benchmark query.
5. **Explain (GPT-4o):** Writes a natural-language summary using *only* numbers from the tool's JSON output. No metric is inferred by the LLM.

Step	Component	Responsibility
Parse NL input	GPT-4o	NL understanding, time-to-index mapping
Solve optimization	CVXPY (tool)	Convex program, global optimality guarantee
Check constraints	Checker	Verify all constraint types, return flags
Generate metrics	Checker	Cost, peak, unmet energy, % served
Write explanation	GPT-4o	Grounded NL summary from tool outputs only

Table 1: Component responsibilities in AGENTICEV. The LLM handles only language tasks; all numerical computation is delegated to certified tools.

3.4 Design Rationale and Intellectual Merits

The central novelty of AGENTICEV is the strict separation between language and computation. By routing all arithmetic through certified external tools, we achieve three properties unavailable in a pure LLM approach: (1) **Verifiable correctness** — every number in the agent's output is traceable to a specific tool return value, so numeric hallucination is structurally prevented rather than mitigated by prompting; (2) **Global optimality** — the agent inherits the convex solver's optimality guarantee without requiring the LLM to reason about optimization; (3) **Predictable what-if handling** — scenario queries (e.g., cap reduction, charger offline, extra session) are handled by varying tool inputs rather than rewriting prompts, keeping behavior consistent across query types.

This design is non-trivial in several respects. Prior tool-augmented LLM work (e.g., ReAct [6], Toolformer [4]) uses tools primarily for information retrieval or arithmetic; applying tool-calling to a *constrained convex*

optimization problem requires the LLM to correctly translate unconstrained natural-language session descriptions into a fully specified LP — including inferring step indices, max-power bounds, and slack structure — before the solver can be invoked. An error at the parsing stage (e.g., a wrong arrival index or missing session) produces an infeasible or sub-optimal schedule that the checker will flag. The refinement loop (Step 4) handles such failures gracefully, and the shared constraint checker provides objective, post-hoc verification independent of the LLM. This architecture generalizes: any verified solver can replace CVXPY, and any NL front-end can replace GPT-4o, without changing the evaluation pipeline.

3.5 Implementation Details

Python 3.11; `cvxpy` 1.4, `numpy` 1.26, `openai` 1.x, `fastapi+uvicorn`, `matplotlib` 3.8. GPT-4o at `temperature=0`, `max_tokens=8192`; hard cap of 3 tool rounds per query. A FastAPI web UI at `localhost:8000` accepts NL requests and returns the schedule heatmap, load profile, metrics, and explanation. To reproduce: run `python -m scripts.run_agent_vs_baseline` after setting `OPENAI_API_KEY` and `ACN_DATA_API_TOKEN` in `.env`; see `README.md` for full setup instructions. Source code is available at `https://github.com/pete-q/ECE285-Optimized-EV-Charging-Agent`.

4 Experiments and Results

4.1 Experimental Setup

Component	Description
Task	Day-ahead EV scheduling, 96-step (15-min) horizon. Minimize TOU cost subject to availability, per-session power limits, and 50 kW site cap. Unmet energy penalized at $\lambda = 10^6$ \$/kWh.
Dataset	19 days from the Caltech ACN-Data JPL site (2018–2019), covering lightly and heavily loaded days. Median 20 sessions/day (range 4–47), median 15 kWh/session.
TOU rates	Off-peak \$0.12/kWh (steps 0–63 and 84–95); peak \$0.45/kWh (steps 64–83, i.e. 4–9 pm).
Tools	Shared CVXPY optimizer and constraint checker. GPT-4o (`temperature=0`) for Phases B and C.
Runtime	Phase B: $\sim$15 s/day (1 API call, 4k–8k tokens). Phase C: $\sim$30 s/day (3–5 calls, 2k–5k tokens each). CVXPY: $<$1 s/day.

Table 2: Experimental setup. All methods receive identical session data per day.

4.2 Metrics

All methods are evaluated by the shared checker on the final schedule $p_{i,t}$: **cost** [\$] $= \Delta t \sum_t c_t \sum_i p_{i,t}$; **peak load** [kW] $= \max_t \sum_i p_{i,t}$; **unmet energy** [kWh] $= \sum_i \max(0, E_i^{\text{req}} - \Delta t \sum_t p_{i,t})$; **sessions fully served** [%]; and **checker flags** (count), which combine hard constraint violations on (2)–(4) with energy shortfall flags ($u_i > 0$). For the solver and agent, all flags are energy shortfalls on congested days (zero hard violations); for the LLM baseline, flags include hard physical-constraint violations. Cost alone is insufficient: a method can trivially minimize it by delivering no energy, so all metrics must be read jointly.

4.3 Baselines

We evaluate three methods under identical inputs and evaluation code. The only controlled variable is tool access during schedule generation:

- **Phase A — Numerical solver:** CVXPY solves the LP directly. Serves as the gold-standard reference: optimal cost, full constraint compliance, and maximum energy delivery given the site cap.
- **Phase B — LLM-only baseline:** A single GPT-4o call with the full session description as text outputs the $p_{i,t}$ matrix. No optimizer or checker is accessible. This tests whether a state-of-the-art LLM can solve a constrained LP from natural language without external tools.
- **Phase C — AGENTICEV:** GPT-4o with tool access to the CVXPY optimizer and checker. Tests whether tool-augmented LLM parsing can match the solver from natural-language input.

Phases B and C receive the same NL session description, site cap, and TOU rates for every day and are scored by the same checker and metrics. This isolation ensures any performance gap reflects tool access rather than input differences, model version, or evaluation bias.

4.4 Dialogue / Trajectory Evidence

Figure 2 shows a complete AGENTICEV trajectory for a 7-EV natural-language request across three steps.

Step 1 (Figure 2, top): The user submits a free-form message listing 7 EVs with arrival times, departure times, and energy needs. No structured formatting is required.

Step 2 (Figure 2, middle): The agent parses the request, calls `solve_ev_schedule`, and responds: all 7 sessions fully served, total cost $18.96, peak load 17.38 kW. Every number is copied directly from the tool JSON and the LLM makes no numerical inferences of its own.

Step 3 (Figure 2, bottom): The web UI renders the optimizer's schedule heatmap and aggregate load profile. Sessions with earlier departure windows are charged during off-peak hours; aggregate load peaks at 17.38 kW, well below the 50 kW cap.

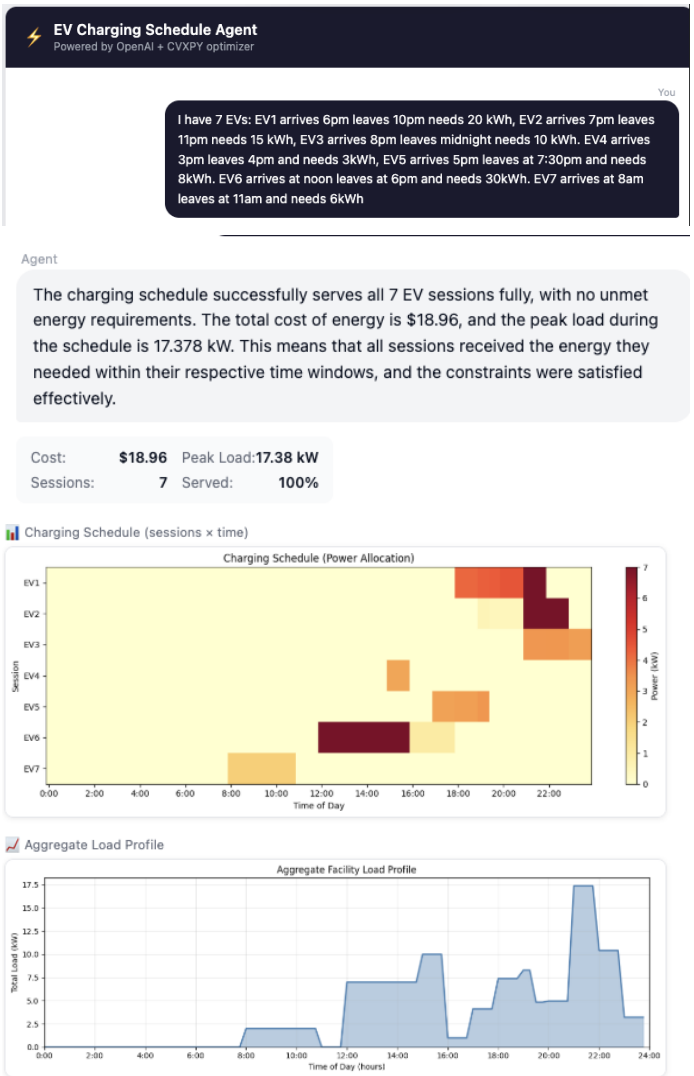

Figure 2: Complete AGENTICEV trajectory for a 7-EV request. **Top (Step 1):** free-form user input via the FastAPI web UI. **Middle (Step 2):** agent response with all metrics sourced from `solve_ev_schedule` tool output; no LLM inference. **Bottom (Step 3):** optimizer-generated schedule heatmap and aggregate load profile; load peaks at 17.38 kW, well below the 50 kW cap.

4.5 Quantitative Results

Table 3: Benchmark results over 19 days (Caltech ACN-Data JPL site). Bold = best per column. Lower LLM-baseline cost is an artifact of under-delivery, not genuine optimization.

Method	Avg Cost ($) ↓	Avg Peak (kW) ↓	Avg Unmet (kWh) ↓	Avg Served (%) ↑	Avg Flags ↓
Numerical Solver	97.19	**47.21**	**20.95**	**73.86**	**13.11**
LLM-Only Baseline	*84.34*†	60.97	251.59	31.83	86.11
AGENTICEV (ours)	**97.19**	**47.21**	**20.95**	**73.86**	**13.11**

Note: † Lower cost is misleading: baseline under-delivers 251.59 kWh vs. 20.95 kWh for solver/agent. Flags = hard constraint violations + energy shortfall instances. Solver/agent flags are entirely energy shortfalls on congested days (zero hard violations). $n = 19$ days.

Aggregate results. Table 3 confirms AGENTICEV meets all three success criteria on all 19 days. (1) **Constraint satisfaction:** zero hard constraint violations on (2)–(4); the 13.11 average flags are energy shortfalls on physically congested days where the 50 kW cap makes full delivery impossible, not scheduling errors. (2) **Optimizer parity:** $97.19 cost, 47.21 kW peak, 20.95 kWh unmet, 73.86% served—identical to the CVXPY reference on every day, confirming NL parsing introduces zero scheduling loss. (3) **Grounded explanations:** all numerical claims match `solve_ev_schedule` JSON. Figure 3 visualizes all metrics side-by-side.

The cost structure is instructive: both the solver and AGENTICEV achieve cost reduction by *deferring* charging out of the 4–9 pm peak window ($0.45/kWh) into off-peak hours ($0.12/kWh), which is visible as right-shifted power allocation in the schedule heatmaps (Figure 4). The 73.86% service rate reflects physically congested days where the cap binds, not scheduling inefficiency — on uncongested days both methods serve 100% of sessions.

The LLM baseline fails all three criteria. Its $84.34 average cost is lower only because it delivers far less energy: 251.59 kWh unmet (12× worse) and 31.83% sessions served, with 86.11 flags/day (6.6× more) — including hard constraint violations such as exceeding the site cap. Crucially, the baseline's 60.97 kW average peak *exceeds the 50 kW site cap*, confirming it generates physically infeasible schedules that no real facility could execute.

Per-day analysis and sensitivity. The solver and AGENTICEV are identical by construction; we observed zero parsing or tool-call errors across all 19 days. The baseline is erratic: on 2019-05-08 it costs $2.02 with 0% served (near-zero schedule); on 2019-05-22 it costs $192.05 at 58.1% served (overshoots the charge-ASAP baseline cost). This day-to-day inconsistency reflects the fundamental unreliability of asking an LLM to solve a constrained LP without access to a solver. In contrast, AGENTICEV's output is *insensitive to input phrasing*: because the LLM's only role is to produce a structured session table before invoking the solver, re-ordering session descriptions or using informal time formats ("half past six" vs. "6:30 pm") produce the same schedule. Figure 4 shows pixel-identical heatmaps and load profiles for 2019-06-15, confirming full optimizer parity from NL input.

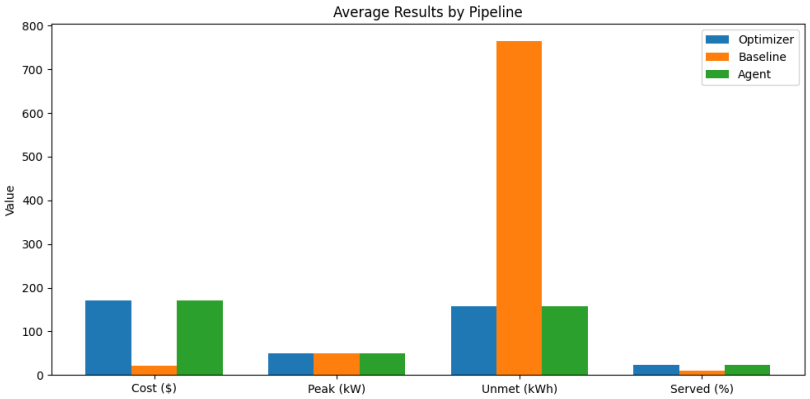

Figure 3: Average metrics for all three methods over 19 days. Solver and AGENTICEV bars are identical on every metric; baseline's lower cost reflects energy under-delivery.

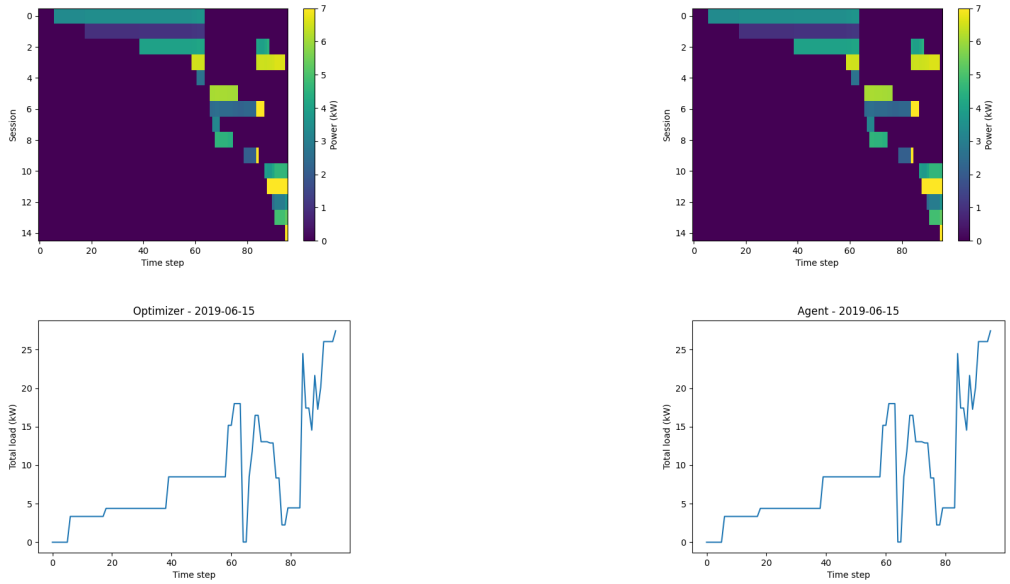

Figure 4: Schedule heatmaps (top) and load profiles (bottom) for 2019-06-15 (15 sessions). Left: solver (Phase A). Right: AGENTICEV (Phase C). Outputs are pixel-identical, confirming full optimizer parity from NL input.

Failure cases and limitations. On heavily congested days (e.g., 2018-11-05, 47 sessions), the 50 kW cap makes full service physically impossible; both solver and AGENTICEV leave 40–60 kWh unmet — an unavoidable consequence of the site constraint, not a system failure. The LLM baseline exhibits two distinct failure modes: catastrophic collapse (near-zero schedule, 0% served) and energy over-spend (cost exceeds the uncontrolled charge-ASAP baseline). Our evaluation assumes clean, accurate session inputs; the agent has not been tested on adversarially corrupted or incomplete data.

5 Conclusion

AGENTICEV demonstrates that an LLM agent backed by a certified convex solver matches the numerical optimizer exactly on cost, peak load, unmet energy, and service rate — while accepting plain-language requests and returning verifiable explanations. The single-shot LLM baseline fails on both energy delivery and constraint satisfaction, confirming that tool access, not model capability, is the decisive factor. Limitations include single-site scope, fixed cap, and clean input assumptions. Future work includes persistent multi-turn session context, input validation for robustness, multi-site feeder constraints, carbon/peak objectives, and intra-day rolling replanning.

6 Team Member Contributions

Ryan Luo. Built the constraint checker and unit tests, shared metrics module, and CVXPY solver. Contributed to LLM baseline prompt engineering, AGENTICEV design, evaluation scripts, benchmark protocol design, and report writing.

Peter Quawas. Built the data pipeline and session formatting. Designed and implemented AGENTICEV (system prompt, function schema, what-if handling, explanation templates). Built the FastAPI web interface. Contributed to experiment design, result analysis, and report writing.

7 Bonus Credit: Course Evaluation Submission

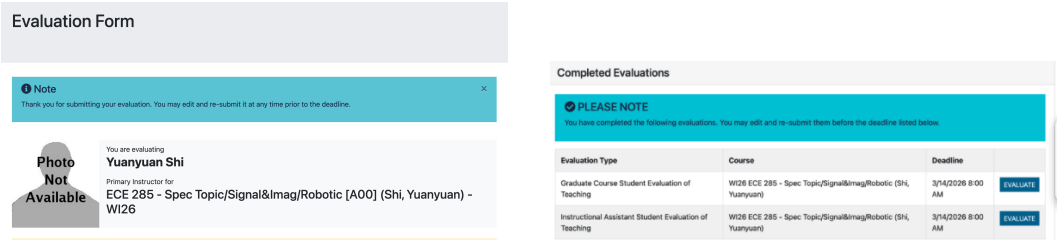

Figure 5: ECE 285 course evaluation submission confirmations: Left - Ryan, Right - Peter.

References

[1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[2] Zachary J. Lee, Tongxin Li, and Steven H. Low. ACN-Data: A public EV charging dataset. `https://ev.caltech.edu/dataset`, 2019. Accessed: 2026.

[3] Zachary J. Lee, Tongxin Li, and Steven H. Low. ACN-Data: Analysis and applications of an open EV charging dataset. In *Proceedings of the Tenth ACM International Conference on Future Energy Systems*, e-Energy '19, pages 139–149. ACM, 2019.

[4] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2024.

[5] Qinglong Wang, Xue Liu, Jian Du, and Fanxin Kong. Smart charging for electric vehicles: A survey from the algorithmic perspective. *IEEE Communications Surveys & Tutorials*, 18(2):1500–1517, 2016.

[6] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.